Développement des composants distribués Web

CHAPITRE 2 :JSP/SERVLET

MVC : Model, Vue, Control

Model : SGBD relationnel

Control : servlet + JSP gérés par un conteneur web

Vue : HTML : client web (léger)

Qu’est ce qu’une Servlet

- Est une API JEE (Jakarta Entreprise Edition)
- Une servlet est un programme Java s'exécutant côté serveur
- Elle reçoit des requêtes de clients (navigateurs web), via le protocole HTTP pour générer une réponse sous la forme de pages web dynamiques
  - HTTP fonctionne selon sur un modèle requête-réponse client-serveur :
    - le navigateur (client) envoie une requête HTTP au serveur (par exemple, pour une URL), en précisant l’action GET, POST, PUT, PATCH, DELETE.
    - PUT remplace intégralement une ressource existante alors que PATCH la modifie partiellement
    - Le serveur la traite et renvoie une réponse HTTP contenant le statut (ex: 200 OK)
- Le navigateur du client affiche la réponse
- Les servlets sont gérées par des conteneurs web comme Tomcat d’apache
- Les servlets fonctionnent en collaboration avec des vues (.jsp)
- Attention le package javax est remplacé par jakarta

Cycle de vie d’une Servlet

Le conteneur Web doit se conformer à la gestion des Servlets :

- création et initialisation de Servlets
- traitement des services demandés par les éventuels clients
- destruction des Servlets et libération de l'espace mémoire (ramasse-miettes)

Java propose une API permettant de construire des Servlets :

- l'API permet au développeur de gérer la création, les requêtes et la destruction des Servlets
- le conteneur fait appel à cette API pour gérer les Servlets

Exemple de Servlet : fichier de déploiment

L'appel à la Servlet se fait par : http://localhost:8080/Servlet/Coucou ou hello-servlet (exemple 2)

web.xml

```
<?xml v e r s i on =" 1.0 " encoding ="ISO -8859 -1 " ?>
<!DOCTYPE web - app
PUBLIC " -// Sun Microsystems , Inc .// DTD Application web 2.3// EN"
" file: /// opt / j2sdkee / conf /web - app_2_3 . dtd ">
<web - app >
<servlet >
<servlet - name >Bonjour </ servlet - name >
<servlet - class >Hello </ servlet - class >
</ servlet >
<servlet - mapping >
<servlet - name >Bonjour </ servlet - name >
<url - pattern >/ Servlet / Coucou </url - pattern >
</ servlet - mapping >
</web - app >
```

Ou

HelloServlet.java

```
@WebServlet(name = "helloServlet", value =
"/hello-servlet")
public class HelloServlet extends HttpServlet {
public void init() {
}
public void doGet(HttpServletRequest request,
HttpServletResponse response) throws
IOException {
    response.setContentType("text/html");
}
public void destroy() {
}}
```

Application web et déploiment

Arborescence d'une archive WAR (Web ARchive)

```
ApplicationWeb.war
  |_ fichiers.html
  |_ fichiers.jsp
  |_ repertoires\fichiers
  |_ src
  |    |_ sourcesServlets.java
  |    |_ sourcesbibliotheques.java
  |_ META-INF
  |    |_ MANIFEST.MF
  |_ WEB-INF
       |_ web.xml
       |_ classes
            |_ Servlets.class
            |_ lib
                |_ bibliotheques.jar
```

le fichier ApplicationWeb est placé dans le répertoire d'applications du conteneur web

- webapps pour Tomcat
- JBOSS_DIR/server/JBOSS_MODE/deploy pour Jboss

API: GenericServlet

API J2EE SDK

Nécessite l'installation du J2EE SDK

Packages : `javax.Servlet.*` et `javax.Servlet.http.*`

La classe `GenericServlet` :

```
public abtract class GenericServlet {
  GenericServlet()
    // methodes gerant l'initialisation et la destruction
  void  destroy()
  void  init()
  void  init(ServletConfig config)
    // methodes pour la recuperation des parametres passes
  String        getInitParameter(String name)
  Enumeration   getInitParameterNames()
    // recuperation d'informations concernant la Servlet
  ServletConfig         getServletConfig()
  ServletContext        getServletContext()
  String        getServletInfo()
  String        getServletName()
    // ecriture de logs, pratique pour le deboguage
  void  log(String msg)
  void  log(String message, Throwable t)
    // traite les requetes (doit etre redefinie dans la sous-classe)
  abstract  void  service(ServletRequest req, ServletResponse res)
}
```

Qu’est ce qu’un JSP

- Un JSP (JavaServer Pages) est une technologie Java côté serveur permettant de créer des pages web dynamiques en intégrant du code Java (placé entre <% %>) directement dans du HTML.
  - Un script JSP est un mélange de HTML, Java et Javascript.
  - Le code Java appelé scriptlet
  - Inconvénient des scriptlets : mélange logique métier et présentation
  - Il est déconseillé de charger un script JSP avec des traitements JAVA lourds.
- Lorsqu'une page .jsp est appelée, le serveur la transforme en une servlet Java, la compile, puis l'exécute pour générer le contenu final qui est envoyé au navigateur du client sous forme de HTML.
- On peut remplacer les scriptlets par l’utilisation d’une bibliothèque de balises (taglib) comme le JSTL (JavaServer Pages Standard Tag Library)

Élément d’un JSP

- Directives
  - **<%@page import="java.util.*" %> : importation de packages java**
  - **<%@taglib prefix="c" uri="WEB-INF/tld/core.tld" %>** importation d’un taglib

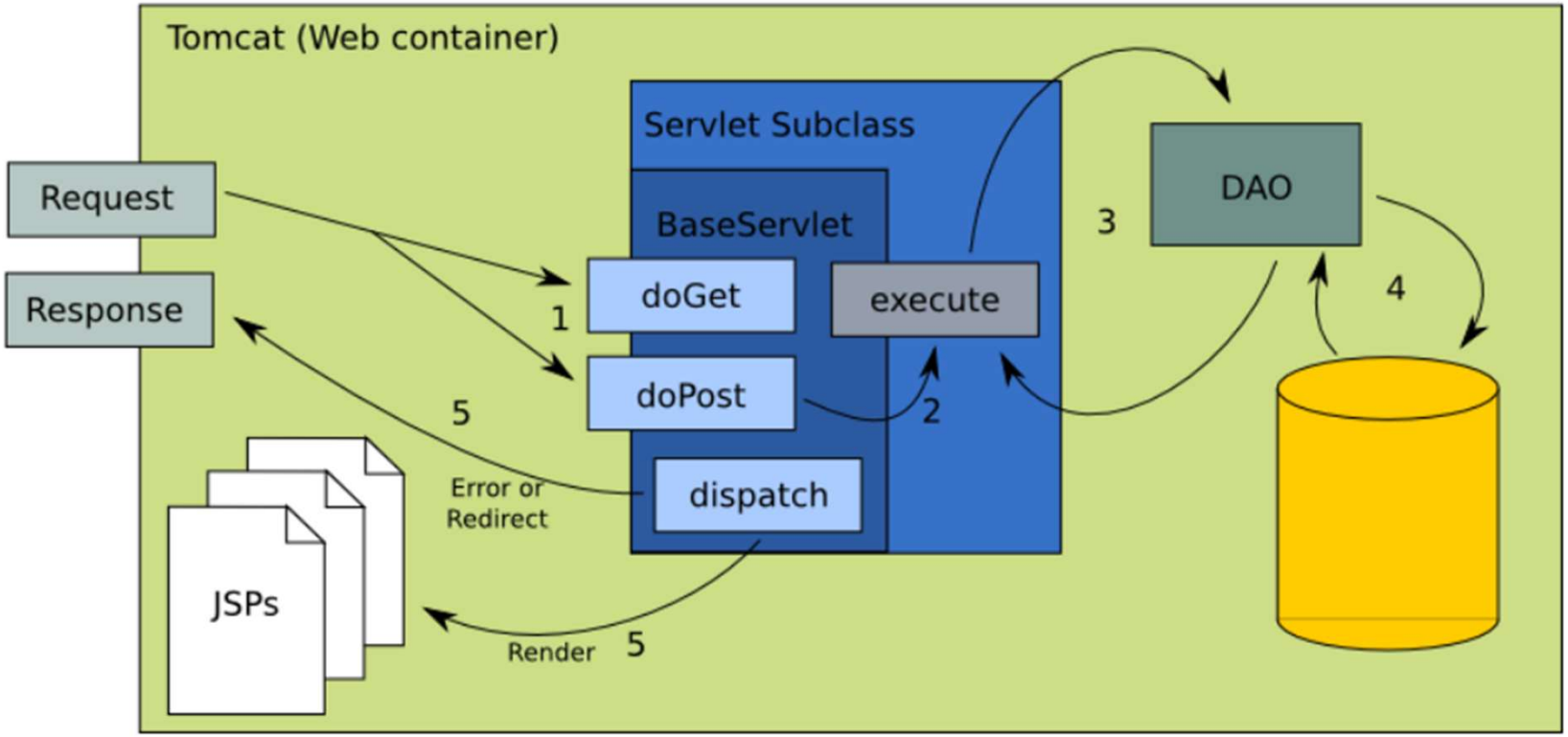
- Elément d’un script
  - ***Scriplets* <% code java %> : traitement java**
  - ***Déclarations* <%! Déclarations %> : déclaration de variables**
  - ***Expressions* <%= expression %> : insérer les messages dans le rendu HTML**
  - request est un objet définit par défaut dans les scriptlets et les expressions sans déclaration

```
<%-- 1. Directive : Définit les paramètres de la page --%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" import="java.util.Date" %>
<html><body>    <h2>Exemple JSP complet</h2>
 <%-- 2. Déclaration : Définit des variables/méthodes de classe (hors méthode de service) --%>
<%!     private String message = "Bienvenue sur la page";
private Date dateActuelle = new Date();    %>
<%-- 3. Scriptlet : Exécute du code Java lors de la requête --%>
<%       String utilisateur = request.getParameter("user");
     if (utilisateur == null) {          utilisateur = "Invité";       }    %>
<%-- 4. Expression : Évalue et insère le résultat dans le HTML --%>
<p>Message: <%= message %></p>
   <p>Utilisateur: <%= utilisateur %></p>
<p>Date : <%= dateActuelle.toString() %></p></body></html>
```

Objets par défaut

- **request** : Représente la requête HTTP (HttpServletRequest) et permet de lire les paramètres, en-têtes et attributs.
- **response** : Représente la réponse HTTP (HttpServletResponse) pour envoyer du contenu au client.
- **session** : Représente la session utilisateur (HttpSession) pour stocker des informations entre plusieurs requêtes.
- **application** : Représente le contexte de l'application (ServletContext), partagé par toutes les pages.
- **out** : Objet utilisé pour écrire du contenu dans le flux de réponse.
- **page** : Référence à l'instance de la servlet courante (équivalent de this)
- **exception** : Accessible uniquement dans une page d'erreur (directive <%@ page isErrorPage="true" %>), contient l'exception levée.

Architecture application MVC (JSP+servlet)



Exemple de Servlet: doPost(1)

Hello.java

```
import java .io .*;
import javax . servlet .*;
import javax . servlet . http .*;
public class Hello extends HttpServlet {
public void doPost ( HttpServletRequest requete , HttpServletResponse reponse ) throws
ServletException , IOException {
String prenom = requete . getParameter (" prenom ");
reponse . setContentType (" text / html ");
PrintWriter out = reponse . getWriter ();
out . println ("<html >");
out . println ("<body >");
out . println ("<h1 > Bonjour " + prenom + " ! </h1 >");
out . println ("<p> Ceci est ma premiere Servlet ... </p>");
out . println (" </body >");
out . println (" </html >");
}}
```

Exemple de Servlet: doPost(2)

Hello.java

```
import java .io .*;
import javax . servlet .*;
import javax . servlet . http .*;
@WebServlet(name = "ajouterServlet", value = "/ajouter-servlet")
public class Hello extends HttpServlet {
protected void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException,IOException {//traitement et renvoie de données vers la JSP

public void doPost ( HttpServletRequest requete , HttpServletResponse reponse ) throws
ServletException , IOException {
// recuperation de données du formulaire et traitement
response.sendRedirect("ajouter-servlet");
}}
```

Exemple de Servlet: doGet(1)

```
Hello.java
import java .io .*;
import javax . servlet .*;
import javax . servlet . http .*;
public class Hello extends HttpServlet {
p u b l i c void doGet ( HttpServletRequest requete , HttpServletResponse reponse ) throws
ServletException , IOException {
request.setAttribute("prenom", requete.getParameter(« prenom"));
RequestDispatcher rd = request.getRequestDispatcher("index.jsp");
rd.forward(request, response);

}}
```

Exemple de Servlet: doGet(2)

index.jsp

```
<td><a href="delete?ref=<%= p.getId() %>">action</a></td>
```

Hello.java

```
import java .io .*;
import javax . servlet .*;
import javax . servlet . http .*;
public class Hello extends HttpServlet {
p u b l i c void doGet ( HttpServletRequest requete , HttpServletResponse reponse ) throws
ServletException , IOException {
int ref=Integer.parseInt(request.getParameter("ref")));
//traitement utilisant ref
}}
```

Paramètres de formulaire

Hello.jsp

```
%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" %>
<head>
< t i t l e >Ma premiere Servlet </ t i t l e >
</head>
<body>
<h1>Bonjour </h1>
<form action =" hello-servlet " method=" post ">
< label >Prenom : < input type =" text " name=" prenom "
s i z e ="30"> </ l a b e l >
< input type =" submit " value =" envoyer ">
</form>
<hr />
</body>
</html >
```

Ou

```
<form action=" hello-servlet " method="post" >
   Nom : <input type="text" name="nom" /><br/>
   Prénom : <input type="text" name="prenom" /><br/>
   Email : <input type="email" name="email" /><br/>
   <input type="submit" value="Ajouter" />
</form>
```

Exemple de code en JSP (java)

index.jsp

```
%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" %>
<%@ page import="java.util.*%>
<!DOCTYPE html>
<html>
<head>
   <title>JSP – Liste produit</title>
</head>
<body>
<h1>Liste des personnes</h1>
<table border="1">
   <tr>
<th>ID</th><th>des</th><th>Prix</th><th>quantite</th>
   </tr>
```

```
<% List<String> personnes = (List<String>)
      request.getAttribute("personnes");
      if(personnes!=null){
      for (String p : personnes) {    %>
   <tr>
      <td><%= p %></td>
</tr>
   <% }} %>
</table>
</body>
</html>
```

Exercice

- Développer une première version "Hello World" constituée d’une servlet et d’un script JSP. La servlet implémente la méthode doGet() qui retourne un message sous la forme "Hello world name" avec name et une variable de type String déclarée et initialisée coté JSP.

- Développer une deuxième version en utilisant un formulaire permettant à un client de saisir son nom et son prénom. La servlet retourne un message d’accueil portant le nom et le prénom du client.

- Développer une troisième version en utilisant un formulaire permettant à un client de saisir son nom et son prénom. La page JSP affiche, dans une table, les noms et prénoms de tous les utilisateurs de la servlet.

DAO : Data Adapter Object

- les servlets manipulent des objets de type DAO : objets qui permettent l'accès aux données à traves des CRUD (méthodes : Create, Read, Update, Delete)
- Les DAO manipulent des Entités (Java Beans) dégagées à partir du diagramme de classes de conception (exemples: Produit, Catégorie, Societe, …)
- Un Java Bean est une classe Java contenant des attributs, des constructeurs et des accesseurs (getter et setter).
- Les DAO manipulent des entités non persistantes
  - Entités stockées sur des vecteur dynamique comme List ou bien sur des collections clés/valeur comme Map
  - Données volatiles
  - List de données avec des CRUD static dans ce cas
- Ou persistantes
  - Connexion avec une base de données
  - Entité en relation avec table (Produit de java projeté sur table Produit)
  - Données non volatiles

Les Java Beans

- Dégagées du diagramme de classes,
- Des entités persistantes : stockées sur une base de données,
- Des données non persistantes : stockées sur des structures de données.

Exercice

Développer une application web qui gère des personnes caractérisées par leurs noms, prénoms et mails. À travers un DAO, l'utilisateur peut ajouter, supprimer et chercher une personne. Les personnes seront gérées comme des objets java non persistantes.